「Cnoi2021」Cirno's Easy Round II Solution

A - 幻想乡程序设计大赛(Contest)

最优化 / 贪心

> 有 n 题，给定每种气球的个数 $\{a_n\}$ 与每题 AC 人数 $\{b_n\}$，求最优的气球安排方案使得发出去的气球尽可能多。
>
> $n \le 10^5$，$a_i, b_i \le 10^4$。

算法一

考虑暴力枚举 $\{b_n\}$ 的全排列，时间复杂度是 $\mathcal{O}(n!)$ 的。

可以通过子任务一。

预期得分：40 points。

算法二

不难发现，如果 $a_1 \le a_2$，$b_1 \le b_2$ 则有：

$$\min\{a_1, b_1\} + \min\{a_2, b_2\} \ge \min\{a_1, b_2\} + \min\{a_2, b_1\}$$

所以可以推广成 min-plus 的排序不等式，所以排好序相加就好了。

答案为 $\sum\limits_{i=1}^{n} \min\{a_i, b_i\}$。

由于出题人很良心的排好了序，时间复杂度为 $\mathcal{O}(n)$。

期望得分 100 points。

证明

设 $f(x) = \min\{a_2, x\} - \min\{a_1, x\} (a_2 \ge a_1)$。$f(x)$ 单调不降。

若 $b_2 \ge b_1$，则 $f(b_2) \ge f(b_1)$，所以

$$\min\{a_2, b_2\} - \min\{a_1, b_2\} \ge \min\{a_2, b_1\} - \min\{a_1, b_1\}$$

变形可得原式。

B - 矩阵(Matrix)

模拟 / 线性代数

> 给定两个长度为 n 的序列 $\{a_n\}$，$\{b_n\}$ 与一个整数 k。
>
> 设矩阵 A 满足 $A_{ij} = a_i \times b_j$，求 A^k 的所有元素的和在模 998244353 意义下的结果。
>
> $n \le 10^5$，$k \le 10^9$。

算法一

根据百科页面给的矩阵乘法的定义进行模拟即可。

时间复杂度 $\mathcal{O}(n^3k)$。

期望得分：10 points。

算法二

发现矩阵乘法具有结合律，所以可以使用快速幂优化矩阵的幂运算。

时间复杂度 $\mathcal{O}(n^3 \log k)$。

期望得分：30 points。

算法三

设 $\vec{a} = [a_1, a_2, \cdots, a_n]$，$\vec{b} = [b_1, b_2, \cdots, b_n]^T$，不难发现 $A = \vec{b}\vec{a}$。（此处为矩阵乘法）

则根据结合律有 $A^k = (\vec{b}\vec{a})^k = \vec{b}(\vec{a}\vec{b})^{k-1}\vec{a}$。

又 $\vec{a}\vec{b} = \vec{a} \cdot \vec{b} = \sum\limits_{i=1}^{n} a_i b_i$，所以 $A^k = \big(\sum\limits_{i=1}^{n} a_i b_i\big)^{k-1} \cdot A$。

而 A 中元素和为 $\sum\limits_{i=1}^{n}\sum\limits_{j=1}^{n} a_i b_j = \big(\sum\limits_{i=1}^{n} a_i\big)\big(\sum\limits_{i=1}^{n} b_i\big)$。

模拟求解即可，时间复杂度 $\mathcal{O}(n + \log k)$。

期望得分：100 points。

C - 区间(Interval)

交互 / 最优化 / DP / 找规律

> 一定次数的询问一个数是在被猜的区间左边、里边还是右边，以猜出这个区间 $[a, b]$。
>
> $1 \le a \le b \le n \le 1500$。

算法一

挨个询问 1 到 n 每个数是否在 $[a, b]$ 内即可。

询问次数为 n。

期望得分：10 points。

算法二

各种二分答案。

考虑其中一种，先根据返回值是 -1 还是 $0,1$ 二分出左端点，再根据返回值是 $-1,0$ 还是 1 二分出右端点。

视实现精细程度，预计询问 $2\lceil\log_2 n\rceil+\epsilon$ 次，一般来说 ϵ 在 0 - 4 之间。

期望得分：40 - 70 points。

算法三

考虑最小化询问次数。

设找到区间 $[1,n]$ 的一个子区间最劣情况最少需要 f_n 次询问。

考虑当前询问点 p。

则有三种情况。

1. p 在区间左边，则该子区间是 $[1,p-1]$ 的子区间，所以 $f_n\leftarrow\max(f_n,f_{p-1}+1)$。
2. p 在区间右边，则该子区间是 $[p+1,n]$ 的子区间，所以 $f_n\leftarrow\max(f_n,f_{n-p}+1)$。
3. p 在区间内
   则被询问的区间为 $[1,p-1]$ 的一个可空的前缀与 $[p+1,n]$ 的一个可空的后缀与 $\{p\}$ 的并。
   设找到 $[1,n]$ 的一个可空前缀最劣情况最少需要 g_n 次询问。
   则 $f_n\leftarrow\max(f_n,g_{p-1}+g_{n-p}+1)$。
   g_n 的递推式类似。

所以最终递推式为：

$$f_n=\min_{i=1}^{n}\left\{\max\{f_{i-1},f_{n-i},g_{i-1}+g_{n-i}\}\right\}+1$$

$$g_n=\min_{i=1}^{n}\left\{\max\{g_{i-1},g_{n-i}\}\right\}+1$$

可以发现 $f_{1500}=20$。

然后记录 DP 时最优决策点进行交互即可。

询问次数为 $\lfloor\log_2 n\rfloor+\lfloor\log_2\frac{4n}{3}\rfloor$。

期望得分：100 points。

算法 Extra

我们发现 $g_n=\lceil\log_2 n\rceil$。这个可以用归纳法证明，

设 $g_n=\lceil\log_2 n\rceil$ 对于所有 $n<2^k$ 成立。

则对任何 $n\in[2^k,2^{k+1})$，有 $g_n=\min\limits_{i=1}^{n}\left\{\max\{g_{i-1},g_{i-p}\}\right\}+1\geq g_{\frac{n-1}{2}}+1=\lceil\log_2 n\rceil$。

所以最优转移点是 $2^{\lceil\log_2 n\rceil-1}$。

然后考虑 f_n。用同样的 dirty work 可以证明 $f_n=\lfloor\log_2 n\rfloor+\lfloor\log_2\frac{4n}{3}\rfloor$。

且最优决策点是 $2^{\lceil\log_2 n\rceil-2}\times\lfloor\frac{n}{2^{\lceil\log_2 n\rceil-2}}\rfloor$。

期望得分：101 points。

D - 符文破译(Spell Words)

最优化 / 字符串 / 贪心 / KMP

> 求字符串 S 最少可以划分成多少个 T 的前缀。
>
> $|\mathtt{S}|, |\mathtt{T}| \le 10^7$。

算法一

设 f_x 为 S 的长度为 n 的前缀最少划分成多少个 T 的前缀。

这显然可以枚举起点 $\mathcal{O}(n)$ 刷表转移。

时间复杂度 $\mathcal{O}(n^2)$。

期望得分：20 points。

算法二

考虑贪心。

从 S 串开头每次匹配最长一段 T 的前缀，直到匹配完为止。

这个贪心是错误的，样例 2 即可 Hack，不过恰好可以通过子任务一。

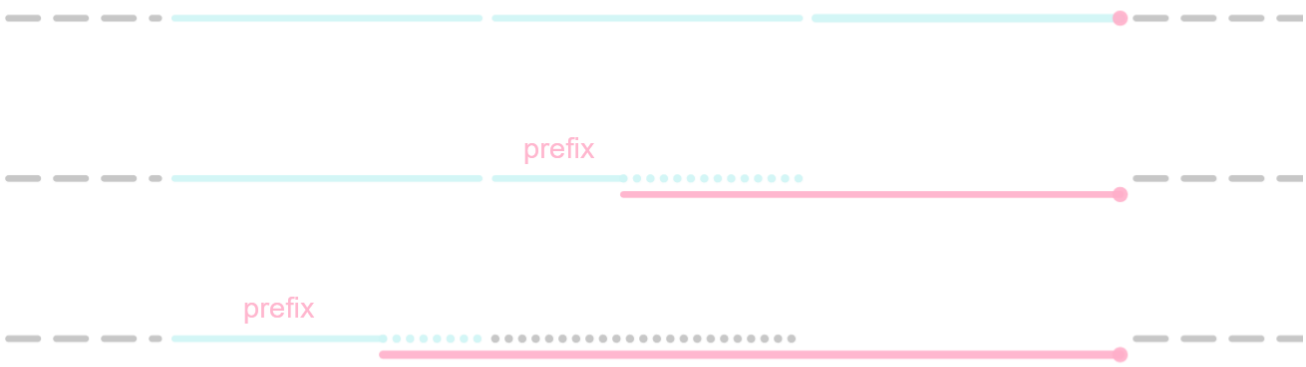
时间复杂度 $\mathcal{O}(n)$。

期望得分：10 points。

算法三

考虑贪心。算法二虽然是错误的，但是，我们发现把它倒过来做就对了。

证明如下：



如图最上边一行代表一种合法的划分方案，而其下两行表示从粉红色点处寻找一个更长的后缀使得它是 T 的前缀，我们发现蓝色段被粉红色段夺走一个后缀后依然是 T 的一个前缀，所以划分依然合法，且划分段数严格不劣。

所以每次从后面选取尽可能长的后缀使得它是 T 的一个前缀一定是最优方案。

证明完贪心的正确性以后，我们发现复杂度却错误了，暴力匹配依然是 $\mathcal{O}(n^2)$ 的。

期望得分：20 points。

算法四

我们发现算法三的本质是找到 S 的一个最长的后缀使得它是 T 的一个前缀。

最长前后缀，我会 KMP！

于是这个贪心便与 KMP 自然地结合起来。

将 T 串与 S 串中夹着一个特殊字符连接起来，KMP 求出每个前缀的 Border。

然后从末尾往前贪心地跳，每次跳的长度是当前前缀的 Border 的长度，直到跳到特殊字符为止。

如果当前 Border 长度为 0，则输出 `Fake`。

时间复杂度 $\mathcal{O}(n)$。

期望得分：100 points。

E - 自我主义的平衡者(Balance)

最优化 / 贪心 / 二分

> n 个人，每个人有预期分数 a_i，将这些人重排列打分，如果前面的人平均分高于或等于预期则打满分，反之则打 0 分，问重排列后最大和最小平均分。
>
> $n, m \le 10^5$。

算法一

枚举全排列，然后模拟打分过程。

时间复杂度 $\mathcal{O}(n!)$。

期望得分：10 points。

算法二

爬山、退火、随机化可能会比较优秀。

期望得分：$10-20$ points。

算法三

发现枚举量太大了，考虑归纳。

如果一个排列中，存在一个人 i 的预期分数比 j 高，且 i 给了 0 分，j 给了满分，交换 i, j 参与评分的顺序，结果没有变化。

但如果按预期分数排序后，相当于把一个左边的满分与右边的 0 分交换分数，如图。

然后我们对排列一直做这种操作直至不能操作，发现最终给 0 分的人一定是按预期分数排序后的一个前缀，而给满分的是剩余的后缀。

所以我们可以枚举前缀，并判断是否有满足条件的排列方式。

构造方式如下：将粉红色的点从小到大压入栈中，蓝色点从大到小压入另一个栈中。

则构造的排列方式是这个双栈的一种出栈序列。

然后构造出栈序列可以贪心，一旦当前平均值小于蓝色栈的栈顶，则弹出蓝色栈，否则弹出粉红色栈。而一旦一个栈为空，而另一继续弹出会导致排列非法，则继续弹出，但是弹出的点染成灰色。

如果贪心的出栈序列没有灰色点就是存在合法的排列，反之则不存在。

证明的话，首先考虑在同时能弹出粉色和蓝色节点时优先弹出蓝色节点不会影响构造的合法性。

如果一个粉色节点与右边相邻的蓝色节点交换位置不影响前缀合法性则交换。

可以知道前缀合法，后缀不变的情况下，整个序列依然合法。

有限次操作后直至不能操作，这种操作称为「下沉操作」。

对序列进行下沉操作后相当于在每个位置优先选择蓝色点。

然后，考虑将产生灰色点的栈重新任意排列后，继续按原来的方式贪心，在原来应当出现灰色节点的时间栈顶的元素不会更优。所以贪心正确。

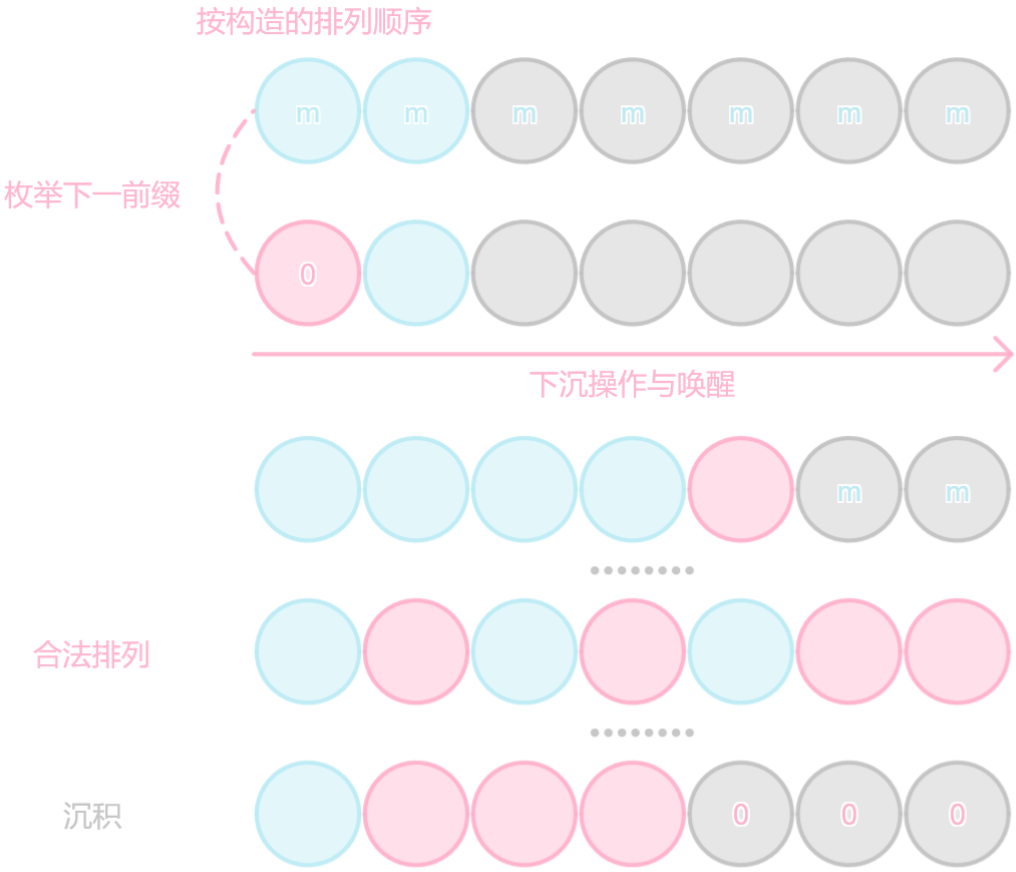
时间复杂度 $\mathcal{O}(n^2)$。

期望得分：50 points。

算法四

发现满足条件的前缀末端是一个区间。

考虑证明。



每次，我们枚举后一个前缀时，相当于把第一个蓝色点变成粉红色然后再做一次「下沉」操作，如图。

设函数 $f(x)$ 表示第 i 个前缀的灰色点个数，其中由蓝色点变成的灰色点记作负数，粉红色记作正数。

由于每次下沉操作可能导致「唤醒」或者「沉积」。所以 $f(x)$ 单调递增。

而我们要 $f(x)=0$ 的 x，这一定是一个区间。

所以用 C 题中任意一种 $\mathcal{O}(\log n)$ 的方法即可找出区间。

时间复杂度 $\mathcal{O}(n\log n)$。

期望得分：100 points。

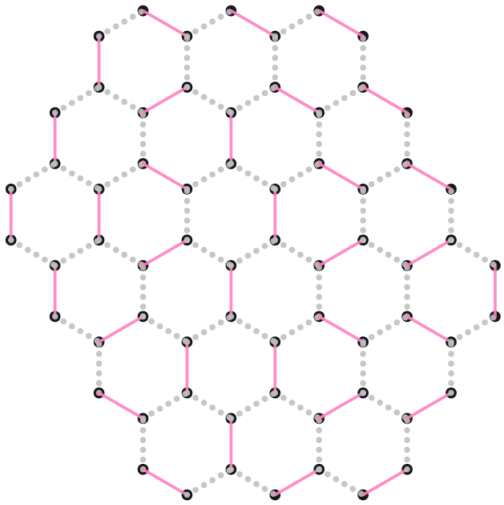
F - 六边形战士(Hexagon)

计数 / 线性代数 / LGV 引理

> 将边长分别为 a, b, c 的平行六边形划分成边长为 1 的等边三角形，求共边三角形连边后形成的图的二分图完美匹配个数。
>
> $a, b, c \le 10^6$。

算法一

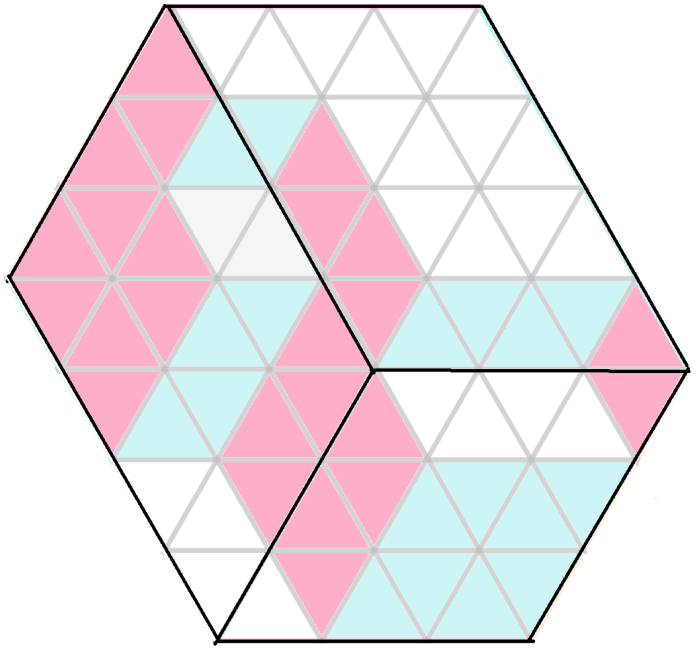
观察生成出来的图和其中的一个完美匹配



样例图

我们发现图中的边分为三种，分别与六边形三组对边垂直。

然后我们将完美匹配中三种不同的边的节点所属的正三角形分别染上粉、白、蓝三种颜色



染色后的结果

我们侧过一个角度，再加上三条辅助线观察，神奇的事情发生了，

这似乎变成了一个立方体的堆叠，而且三种颜色恰好投影到三个侧面上。

所以我们推测完美匹配都与 $a \times b \times c$ 的空间中的凸立方体堆叠一一对应。

具体的证明可以参考 Solara570 的视频。

所以问题转化为凸立方体堆叠计数，考虑一种凸堆叠从上往下看每个位置上的方块数，比如

$$\begin{matrix} 3 & 3 & 3 \\ 2 & 3 & 3 \\ 0 & 3 & 3 \\ 0 & 2 & 2 \end{matrix}$$

我们发现方块数满足一种单调性。

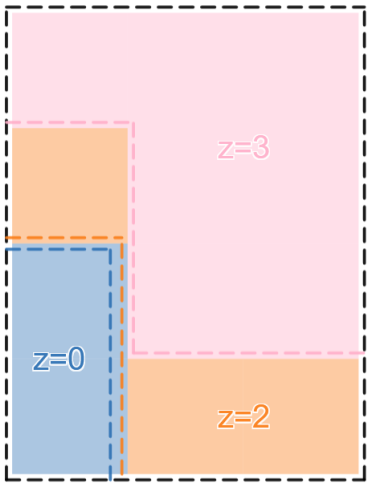
所以问题等价于在一个 $a \times b$ 的矩阵 A 内填数，使得 $0 \le A_{i,j} \le c$，$A_{i,j} \le A_{i+1,j}$ 且 $A_{i,j} \le A_{i,j+1}$ 的方案数。

所以我们可以暴力枚举填数方案，时间复杂度为 $\mathcal{O}(c^{a \times b})$。

期望得分：10 points。

算法二

考虑算法一中的矩阵 A。我们将它中间不同的数组成的连通块的交界处连线，特别地没有出现过的数字也有一个面积为 0 的连通块。



我们发现图中总共有 c 条从左上到右下的路径，可以编号为 $1-n$ 。每条路径每步都只能向右或向下。

而且编号小的路径不能越过编号大的路径到它右上但可以重合。

所以我们考虑路径的方案。

将所有可能的路径按照是否包含建立有向无环图，然后在 DAG 上可重地选取一条链上的 c 个点。

显然可以 DP。

因为 $\binom{2n}{n} \approx \frac{4^n}{\sqrt{\pi n}}$，设 $n=\max\{a,b\}$，时间复杂度为 $\mathcal{O}(\frac{8^n}{n})$。

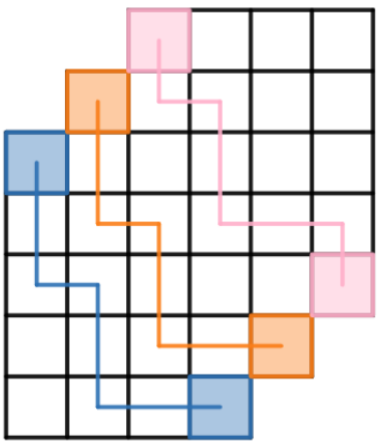
期望得分：20 points。

算法三

路径不能越过，很容易让人想到 LGV 引理。

但是 LGV 定理并不允许路径重合的情况。

所以我们将编号为 i 的路径向上向右平移 i 个单位即可满足不相交的条件。



然后我们设 $\omega(x,y)$ 表示从 x 号起点到 y 号终点的方案数。

很容易知道 $\omega(x,y)=\binom{a+b}{a+x-y}$。

设矩阵

$$M=\begin{pmatrix}\omega(1,1) & \omega(1,2) & \cdots & \omega(1,c)\\ \omega(2,1) & \omega(2,2) & \cdots & \omega(2,c)\\ \vdots & \vdots & \ddots & \vdots\\ \omega(c,1) & \omega(c,2) & \cdots & \omega(c,c)\end{pmatrix}$$

而根据 LGV 引理方案数为 $\det(M)$。LGV 的证明可以参考洛谷题解。

高斯消元求解即可，时间复杂度为 $\mathcal{O}(n^3+m)$，$n=\min\{a,b,c\}$，$m=\max\{a,b,c\}$。

期望得分：90 points。

算法四

喜闻乐见的推柿子时间。

$$\begin{aligned}\det(M)&=\prod_{i=1}^{c}\frac{(a+b)!}{(a+c-i)!(b+i-1)!}\det\left(\prod_{k=j+1}^{c}(a+k-i)\prod_{k=2}^{j}(b+i-k+1)\right)_{i,j=1}^{c}\\&=\prod_{i=1}^{c}\frac{(a+b)!}{(a+c-i)!(b+i-1)!}\det\left((-1)^{j-1}\prod_{k=j+1}^{c}(-i+a+k)\prod_{k=2}^{j}(-i+k-b-1)\right)_{i,j=1}^{c}\\&=(-1)^{c(c-1)/2}\prod_{i=1}^{c}\frac{(a+b)!}{(a+c-i)!(b+i-1)!}\det\left(\prod_{k=j+1}^{c}(-i+a+k)\prod_{k=2}^{j}(-i+k-b-1)\right)_{i,j=1}^{c}\end{aligned}$$

然后设 $x_n=-n$，$b_n=a+n$，$a_n=n-b-1$，根据 Krattenthaler's formula

$$\det\left(\prod_{k=2}^{j}(x_i+a_k)\prod_{k=j+1}^{n}(x_i+b_k)\right)_{i,j=1}^{n}=\prod_{1\le i<j\le n}(x_i-x_j)\prod_{2<i\le j\le n}(a_i-b_j)$$

有

$$\begin{aligned}\det(M)&=\prod_{i=1}^{c}\frac{(a+b)!}{(a+c-i)!(b+i-1)!}\prod_{1\le i<j\le c}(j-i)\prod_{2\le i\le j\le c}(a+b+j-i+1)\\&=\prod_{i=1}^{c}\frac{(a+b)!}{(a+c-i)!(b+i-1)!}\prod_{i=1}^{c-1}(i!)\prod_{i=1}^{c-1}\frac{(a+b+i)!}{(a+b)!}\\&=\prod_{i=1}^{c-1}(i!)\prod_{i=1}^{c}\frac{1}{(a+i-1)!}\prod_{i=1}^{c}\frac{1}{(b+i-1)!}\prod_{i=1}^{c-1}\frac{(a+b)!}{(a+b)!}\prod_{i=1}^{c-1}(a+b+i)!\times(a+b)!\\&=\prod_{i=1}^{c-1}(i!)\times\frac{\prod_{i=1}^{a-1}(i!)}{\prod_{i=1}^{a+c-1}(i!)}\times\frac{\prod_{i=1}^{b-1}(i!)}{\prod_{i=1}^{b+c-1}(i!)}\times\frac{\prod_{i=1}^{a+b+c-1}(i!)}{\prod_{i=1}^{a+b-1}(i!)}\end{aligned}$$

设 $H(n)=\prod_{i=1}^{n-1}(i!)$，则有

$$\det(M)=\frac{H(a)H(b)H(c)H(a+b+c)}{H(a+b)H(b+c)H(a+c)}$$

可以 $\mathcal{O}(a+b+c)$ 求解。

期望得分：100 points。

证明 Krattenthaler's formula

首先引入几个定义。

对称多项式(symmetric polynomial)

- 若 $f(x_1, x_2, \cdots, x_n)$ 是一个 n 元多项式，σ 是任意大小为 n 的排列，$f\left(x_{\sigma(1)}, x_{\sigma(2)}, \cdots, x_{\sigma(n)}\right) = f(x_1, x_2, \cdots, x_n)$，则称 f 是一个对称多项式。

交错多项式(alternating polynomial)

- 若 $f(x_1, x_2, \cdots, x_n)$ 是一个 n 元多项式，σ 是任意大小为 n 的排列，$f\left(x_{\sigma(1)}, x_{\sigma(2)}, \cdots, x_{\sigma(n)}\right) = (-1)^{\mathcal{I}(\sigma)} f(x_1, x_2, \cdots, x_n)$，其中 $\mathcal{I}(\sigma)$ 表示 σ 的逆序对个数，则称 f 是一个交错多项式。

范德蒙乘积(Vandermonde product)

- 若 $f(x_1, x_2, \cdots, x_n) = \sum\limits_{1 \le i < j \le n} (x_i - x_j)$，则称 f 是范德蒙乘积，显然 f 是交错多项式。

然后我们有

命题 1 如果 $f(x_1, x_2, \cdots, x_n)$ 是一个 d 阶的交错多项式，则有

$$g(x_1, x_2, \cdots, x_n) = \frac{f(x_1, x_2, \cdots, x_n)}{\prod\limits_{1 \le i < j \le n} (x_i - x_j)}$$

是一个 $d - \frac{n(n-1)}{2}$ 阶的对称多项式。

考虑证明，对于任意 $1 \le i < j \le n$。将 f 视作 x_i 的多项式，然后带入 $x_i = x_j$，我们知道

$$f(\cdots, x_{i-1}, x_j, x_{i+1}, \cdots, x_j, \cdots) = -f(\cdots, x_{i-1}, x_j, x_{i+1}, \cdots, x_j, \cdots) = 0$$

所以 f 一定含有 $(x_i - x_j)$ 这个因子。

所以 f 可以整除 $\prod\limits_{1 \le i < j \le n} (x_i - x)j$，$g$ 是多项式，且它的对称性显然。

Krattenthaler's formula 的证明

设

$$f(x_1, x_2, \cdots, x_n) = \det \left(\prod_{k=2}^{j} (x_i + a_k) \prod_{k=j+1}^{n} (x_i + b_k) \right)_{i,j=1}^{n}$$

根据行列式的性质可知 f 是交错多项式。

根据命题 1，我们可以知道

$$f(x_1, x_2, \cdots, x_n) = g(x_1, x_2, \cdots, x_n) \prod_{1 \le i < j \le n} (x_i - x_j)$$

其中 g 是对称多项式。

更具行列式的展开方式，对任何一个变量 x_i 来说，f 是 x_i 的一个 $n-1$ 次多项式

而范德蒙乘积恰好也是一个 $n-1$ 次多项式，所以 g 与 x_i 无关。

所以 g 是一个常数。

$$\det \left(\prod_{k=2}^{j} (x_i + a_k) \prod_{k=j+1}^{n} (x_i + b_k) \right)_{i,j=1}^{n} = C \prod_{1 \le i < j \le n} (x_i - x_j)$$

C 只与 a_i, b_i 有关。

我们不妨设 $x_i = -b_i$ 使得行列式变成上三角的形式，容易求出

$$C = \prod_{2 \le i \le j \le n} (a_i - b_j)$$

所以我们有 Krattenthaler's formula

$$\det \left(\prod_{k=2}^{j} (x_i + a_k) \prod_{k=j+1}^{n} (x_i + b_k) \right)_{i,j=1}^{n} = \prod_{1 \le i < j \le n} (x_i - x_j) \prod_{2 < i \le j \le n} (a_i - b_j)$$

Q.E.D